

# Trail Dades <=> Kech

# Project C++

*APPLICATION DE JEU DE CACHETTE*

Le rapport

**Module : POO en C++**

**Filières : INF-S4 & IAP-S4**

**Prof: R HANNANE**

**Année universitaire 2025/2026**

Prepare par :

**EL ABDELLAOUI M'HAMED**

**2331944**

**GR-02-IAP-S4**

**RANIA LHLOU**

**2415087**

**GR-03-IAP-S4**





# I. Contents

|       |                                              |    |
|-------|----------------------------------------------|----|
| I.    | Introduction :.....                          | 3  |
|       | Objectifs pédagogiques .....                 | 3  |
| II.   | Présentation du Trail de Tinghir.....        | 4  |
|       | Caractéristiques du parcours fictive .....   | 4  |
| III.  | FONCTIONNEMENT GLOBAL DU JEU .....           | 5  |
| A.    | Mécaniques de jeu .....                      | 5  |
| B.    | Capture d'écran du gameplay.....             | 5  |
| IV.   | ARCHITECTURE DÉTAILLÉE DU JEU .....          | 6  |
| V.    | ANALYSE TECHNIQUE DE LA CLASSE GAME.....     | 6  |
|       | Limitations observées.....                   | 6  |
| VI.   | ANALYSE DU JOUEUR (PLAYER) .....             | 7  |
|       | États possibles du joueur .....              | 7  |
| VII.  | ANALYSE DES OBSTACLES .....                  | 8  |
|       | Avantages de l'Object Pool .....             | 8  |
|       | Limites actuelles .....                      | 8  |
| VIII. | GESTION DU BACKGROUND .....                  | 9  |
|       | Principe du défilement infini.....           | 9  |
|       | Point d'attention : usage de const_cast..... | 9  |
| IX.   | INTERFACE UTILISATEUR (HUD) .....            | 10 |
|       | Éléments affichés .....                      | 10 |
|       | Limites identifiées.....                     | 10 |
| X.    | ERREURS ET LIMITES IDENTIFIÉES .....         | 11 |
| XI.   | AMÉLIORATIONS POSSIBLES.....                 | 12 |
| a.    | Qualité du code .....                        | 12 |
| b.    | Gameplay .....                               | 12 |
| c.    | Expérience utilisateur.....                  | 12 |
| XII.  | CONCLUSION.....                              | 13 |
|       | Compétences acquises .....                   | 13 |
| XIII. | Les roussource .....                         | 14 |
|       | La bibliothèque : .....                      | 14 |
|       | Les images .....                             | 14 |
|       | le joueur : .....                            | 15 |

# I. Introduction :

Dans le cadre de notre formation en développement informatique, nous avons été amenés à concevoir et réaliser un projet complet consistant en la création d'un **jeu vidéo en deux dimensions (2D)** utilisant le langage C++ et la bibliothèque graphique **SFML**. Ce projet vise à mettre en pratique les concepts fondamentaux de la programmation orientée objet, tout en nous confrontant à des problématiques concrètes liées au développement d'applications interactives en temps réel.

Afin de donner une dimension originale et réaliste à notre travail, nous avons choisi de nous inspirer d'un événement réel : **le Trail de Tinghir**, une course sportive organisée dans la région des gorges et des oasis au Maroc. Le trail est une discipline exigeante qui se déroule en milieu naturel et qui met à l'épreuve l'endurance et la capacité d'adaptation des participants. L'édition 2026, correspondant à la deuxième édition de cet événement, nous a servi de base pour structurer notre jeu autour de deux environnements distincts : **les oasis de Dadès** pour le premier niveau et **la ville de Marrakech** pour le second.

L'objectif n'était pas uniquement de créer un jeu fonctionnel, mais également de concevoir une architecture logicielle cohérente, d'optimiser la gestion des ressources et d'assurer une interaction fluide avec l'utilisateur. Tout au long du développement, nous avons rencontré plusieurs difficultés, notamment dans la gestion des collisions, l'organisation du code et la synchronisation des éléments du jeu, ce qui nous a permis d'approfondir notre compréhension des systèmes temps réel.

## Objectifs pédagogiques

- Maîtriser les concepts de la POO en C++ : héritage, encapsulation, polymorphisme.
- Découvrir une bibliothèque graphique professionnelle (SFML 2.6).
- Concevoir une architecture logicielle modulaire et maintenable.
- Gérer les boucles temps réel, les événements et les collisions.
- Travailler en binôme avec gestion de versions et répartition des tâches.

## II. Présentation du Trail de Tinghir

Le Trail de Tinghir est une compétition sportive de course nature organisée dans le sud-est du Maroc, au cœur de la province de Tinghir. La région se distingue par ses paysages spectaculaires : gorges profondes, oasis verdoyantes du Dadès, kasbahs en pisé et reliefs de l'Atlas. La deuxième édition, prévue en 2026, attire des coureurs marocains et internationaux passionnés par les défis d'endurance en milieu naturel.

Notre jeu rend hommage à cet événement en proposant deux niveaux directement inspirés du parcours : le joueur traverse d'abord les oasis de Dadès, puis rejoint la ville historique de Marrakech. Chaque environnement possède son propre décor, ses obstacles et son ambiance visuelle.



**Niveau 1** — Oasis de Dadès



**Niveau 2** — Ville de Marrakech

### Caractéristiques du parcours fictive

| Niveau | Environnement      | Difficulté              | Durée  |
|--------|--------------------|-------------------------|--------|
| 1      | Oasis de Dadès     | Facile — initiation     | ≈ 30 s |
| 2      | Ville de Marrakech | Élevée — vitesse accrue | ≈ 40 s |

### III. FONCTIONNEMENT GLOBAL DU JEU

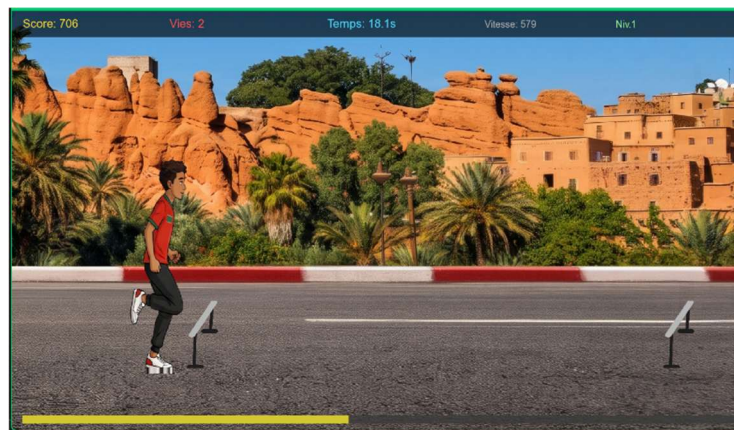
Le jeu développé repose sur un principe simple mais évolutif : il s'agit d'un jeu de course en 2D dans lequel le joueur doit éviter des obstacles tout en survivant pendant une durée déterminée.

Le gameplay est basé sur une progression en deux niveaux. Le premier niveau, situé dans les oasis de Dadès, sert d'introduction et permet au joueur de se familiariser avec les mécanismes du jeu. Le second niveau, se déroulant à Marrakech, introduit une difficulté plus élevée, notamment à travers une augmentation de la vitesse et une fréquence plus importante des obstacles.

#### A. Mécaniques de jeu

|                  |                                                            |
|------------------|------------------------------------------------------------|
| ■ <b>Sauter</b>  | Espace / Flèche haut — pour franchir un obstacle au sol.   |
| ■ <b>Glisser</b> | Flèche bas — pour passer sous un obstacle aérien.          |
| ■ <b>Pause</b>   | Touche P — interrompt la partie sans perte de progression. |
| ♥ <b>Vies</b>    | 3 vies au démarrage ; chaque collision en retire une.      |
| ■ <b>Temps</b>   | Compte à rebours indiqué dans le HUD.                      |

#### B. Capture d'écran du gameplay



Capture du jeu — premier niveau (oasis de Dadès)

## IV. ARCHITECTURE DÉTAILLÉE DU JEU

L'architecture du projet repose sur plusieurs classes principales, chacune ayant un rôle spécifique dans le fonctionnement global.

La classe Game constitue le cœur du système. Elle gère la fenêtre graphique, la boucle principale, les états du jeu (menu, jeu, pause, fin), ainsi que les interactions entre les différents composants. Elle agit comme un contrôleur central.

La classe Player représente le personnage principal. Elle gère les déplacements, les états (course, saut, glissade), les animations ainsi que les interactions avec les obstacles.

La classe ObstacleManager est responsable de la génération et de la gestion des obstacles. Elle utilise un système d'Object Pool, permettant de gérer efficacement les objets sans recréation constante.

La classe Background s'occupe de l'affichage du décor et implémente un système de défilement infini pour simuler le mouvement.

Enfin, la classe HUD permet d'afficher les informations essentielles du jeu, telles que le score, les vies, le temps restant et la vitesse.

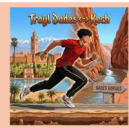
## V. ANALYSE TECHNIQUE DE LA CLASSE GAME

La classe Game est responsable de la boucle principale du jeu. Cette boucle suit une structure classique composée de trois étapes : la gestion des événements, la mise à jour et le rendu.

Cette organisation est efficace et permet de maintenir une exécution fluide. Cependant, nous avons constaté que cette classe regroupe un grand nombre de responsabilités, ce qui la rend difficile à maintenir. Elle gère à la fois la logique du jeu, l'interface utilisateur, les entrées clavier et les transitions entre états.

### Limitations observées

Cette classe regroupe un grand nombre de responsabilités, ce qui la rend difficile à maintenir. Elle gère à la fois la logique du jeu, l'interface utilisateur, les entrées clavier et les transitions entre états. Cela constitue une limite importante : une meilleure séparation des responsabilités (pattern State, gestionnaire d'écrans dédié) permettrait d'améliorer la lisibilité et la modularité du code.



## VI. ANALYSE DU JOUEUR (PLAYER)

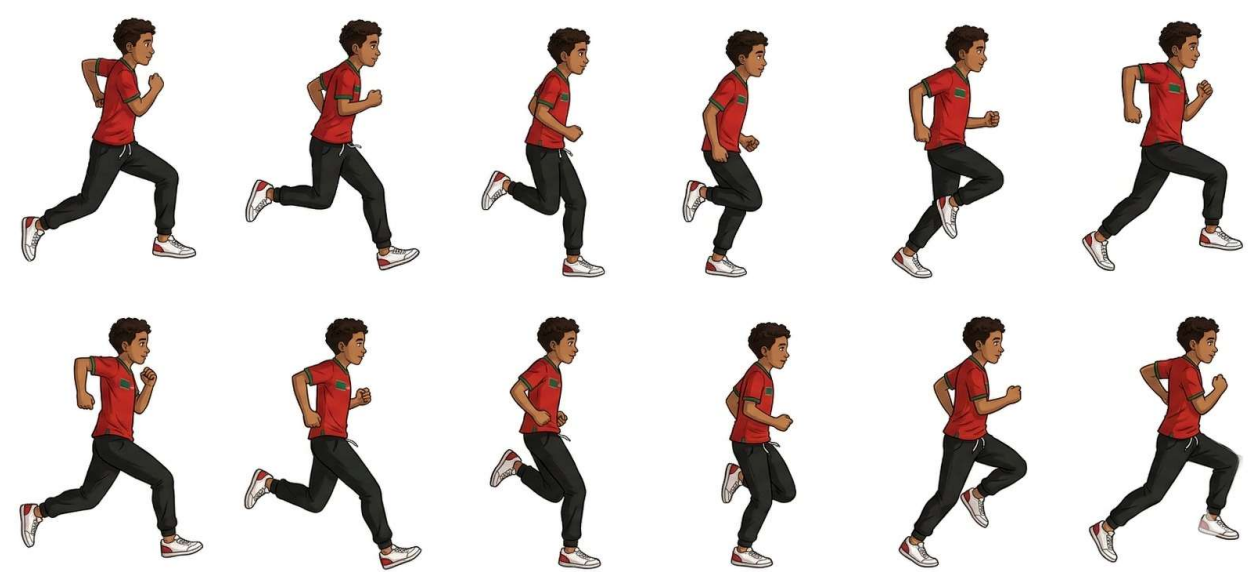
La classe Player implémente le comportement du personnage principal. Elle repose sur un système d'états permettant de distinguer les différentes actions : courir, sauter et glisser.

### États possibles du joueur

| État         | Description                            | Transition                  |
|--------------|----------------------------------------|-----------------------------|
| <b>RUN</b>   | Course au sol, animation cyclique.     | État par défaut.            |
| <b>JUMP</b>  | Saut avec gravité simulée.             | RUN → JUMP via Espace.      |
| <b>SLIDE</b> | Glissade pour passer sous obstacle.    | RUN → SLIDE via flèche bas. |
| <b>HIT</b>   | Collision détectée, bref clignotement. | Tout état → HIT → RUN.      |

La gestion de la gravité est correctement implémentée, ce qui permet de simuler un mouvement réaliste lors des sauts. Le système d'animation basé sur des frames donne également un aspect dynamique au personnage.

Cependant, la détection des collisions repose sur des dimensions fixes définies manuellement, ce qui peut ne pas correspondre exactement au sprite affiché. Cela peut entraîner des incohérences visuelles lors des interactions avec les obstacles.



Cycle d'animation du joueur (frames générées par IA)

## VII. ANALYSE DES OBSTACLES

Le système d'obstacles est géré par la classe ObstacleManager, qui utilise un Object Pool pour optimiser la gestion mémoire.

### Avantages de l'Object Pool

- ✓ Réduction des allocations dynamiques (moins de pression sur le tas).
- ✓ Meilleure stabilité du framerate (pas de pics dus au garbage).
- ✓ Code plus prévisible : nombre d'obstacles borné.

Les obstacles sont générés de manière aléatoire, ce qui rend le jeu plus dynamique et moins prévisible. On distingue plusieurs types d'obstacles, notamment les obstacles au sol et les obstacles aériens.

### Limites actuelles

La détection des collisions repose sur des **ajustements manuels** des hitboxes, ce qui montre que le système n'est pas totalement précis. Une amélioration consisterait à utiliser des collisions basées sur les sprites réels (pixel-perfect) ou sur des polygones englobants plus fins (AABB ajustés automatiquement à partir du sprite).



Exemple d'obstacles dans le niveau



## VIII. GESTION DU BACKGROUND

Le décor du jeu est géré à l'aide de **deux sprites** qui défilent en continu. Lorsqu'un sprite sort de l'écran à gauche, il est immédiatement repositionné à droite du second, créant ainsi une boucle infinie. Cette technique classique permet de créer un effet de mouvement fluide sans interruption visuelle ni saut d'image.

Le système prévoit également un mode de secours en cas d'absence de texture, ce qui constitue une bonne pratique.

### Principe du défilement infini

```
if (sprite1.position.x + sprite1.width < 0) {    sprite1.position.x =  
sprite2.position.x + sprite2.width; }
```

Le système prévoit également un **mode de secours** en cas d'absence de texture (rectangle de couleur unie), ce qui constitue une bonne pratique pour éviter les plantages liés à des fichiers manquants.

### Point d'attention : usage de `const_cast`

L'utilisation de `const_cast` dans cette classe est **problématique** : elle viole les règles de constance du langage C++ et peut entraîner des comportements imprévisibles. La solution propre consiste à revoir la conception (méthode mutable ou suppression du `const` initial) plutôt que de contourner le système de types.



Décors des deux niveaux — défilement infini horizontal



# IX. INTERFACE UTILISATEUR (HUD)

Le HUD (Head-Up Display) permet d'afficher les informations importantes du jeu en temps réel. Il inclut le score, le nombre de vies, le temps restant et une barre de progression visuelle. L'objectif est d'informer le joueur sans encombrer l'écran de jeu.

## Éléments affichés

| Élément              | Position       | Mise à jour             |
|----------------------|----------------|-------------------------|
| Score                | Haut gauche    | À chaque obstacle évité |
| Vies (♥)             | Haut centre    | À chaque collision      |
| Temps                | Haut droite    | Chaque frame            |
| Barre de progression | Bas écran      | Continue                |
| Vitesse              | Coin inférieur | Augmente avec le niveau |

L'implémentation est bien réalisée, notamment grâce à l'utilisation de **couleurs dynamiques** qui évoluent en fonction du temps restant : vert au début, jaune lorsqu'il reste moins de 30 secondes, rouge clignotant en fin de partie. Cela améliore considérablement l'expérience utilisateur.

## Limites identifiées

Certains calculs ne prennent pas en compte des **cas extrêmes**, comme une division par zéro lorsque la durée totale est nulle (ce qui ne devrait normalement pas se produire mais reste possible par erreur de configuration). Il conviendrait d'ajouter des assertions et des valeurs par défaut défensives.



## X. ERREURS ET LIMITES IDENTIFIÉES

Au cours du développement, plusieurs problèmes ont été identifiés. Les recenser de façon transparente est essentiel pour planifier les évolutions futures et apprendre de nos choix de conception.

| Problème                       | Description                                                                                          |
|--------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Pointeurs bruts</b>         | L'utilisation de new/delete manuels présente un risque de fuite mémoire en cas d'exception.          |
| <b>const_cast</b>              | Mauvaise pratique qui peut compromettre la stabilité du programme et trahit un défaut de conception. |
| <b>Classe Game volumineuse</b> | Trop de responsabilités concentrées : maintenance, tests et évolutions deviennent difficiles.        |
| <b>Collisions imprécises</b>   | Hitboxes ajustées à la main, parfois décalées par rapport au sprite réel.                            |
| <b>Absence de sons</b>         | Aucun retour sonore pour les sauts, collisions ou changements de niveau.                             |
| <b>Pas de sauvegarde</b>       | Le meilleur score n'est pas conservé entre deux sessions.                                            |

## XI. AMÉLIORATIONS POSSIBLES

Plusieurs améliorations peuvent être envisagées pour faire évoluer le projet vers une version plus robuste et plus riche. Nous les regroupons en trois catégories : qualité du code, gameplay, et expérience utilisateur.

### a. Qualité du code

- Remplacer les pointeurs bruts par des smart pointers (`std::unique_ptr`, `std::shared_ptr`).
- Supprimer les `const_cast` en revoyant la conception des classes concernées.
- Découper la classe `Game` (pattern `State`, `GameStateManager` dédié).
- Ajouter des tests unitaires sur la logique de collision et de score.

### b. Gameplay

- Mettre en place un système de collisions plus précis (pixel-perfect ou AABB ajustés).
- Introduire des bonus (étoiles, vies supplémentaires, ralentissement temporaire).
- Ajouter un troisième niveau (ex. : médina de Marrakech la nuit).
- Système de difficulté progressive piloté par le score.

### c. Expérience utilisateur

- Intégrer des effets sonores (sauts, collisions, ambiance) et une musique de fond.
- Ajouter un menu d'options (volume, langue, contrôles).
- Sauvegarder le meilleur score localement.
- Animations de transition entre les écrans.

## XII. CONCLUSION

Ce projet nous a permis de mettre en pratique nos connaissances en programmation orientée objet et de mieux comprendre les enjeux liés au développement de jeux vidéo. Il nous a également permis de nous confronter à des problèmes concrets, notamment en termes d'organisation du code, de gestion des ressources et de synchronisation des interactions en temps réel.

Au-delà de l'aspect purement technique, le choix d'un thème inspiré du Trail de Tinghir 2026 nous a donné l'occasion de valoriser un événement sportif marocain et de travailler une direction artistique cohérente, depuis les décors jusqu'aux animations du personnage.

Malgré certaines difficultés et limites identifiées, ce projet constitue une expérience très enrichissante et représente une base solide pour des développements futurs plus avancés. Les améliorations listées au chapitre précédent constituent une feuille de route claire pour faire évoluer le jeu vers une version plus aboutie.

### Compétences acquises

- Maîtrise pratique des concepts POO : classes, héritage, encapsulation, polymorphisme.
- Découverte d'une bibliothèque graphique professionnelle (SFML 2.6).
- Conception d'une boucle temps réel et gestion des événements.
- Optimisation mémoire (Object Pool) et structuration d'un projet C++.
- Travail en binôme : répartition des tâches et résolution collaborative de bugs.



## XIII. Les roussource

### La bibliothèque :

#### SFML 2.6.0

Simple and Fast Multimedia Library

Bibliothèque C++ multiplateforme (Windows, macOS, Linux) fournissant des modules pour le graphique, l'audio, le réseau et la gestion des fenêtres.

### Les images

Les fonds des deux niveaux ont été générés à l'aide de **Google Gemini** à partir de descriptions textuelles inspirées des paysages réels de la région de Tinghir et de Marrakech.

Background level 1 Cree par Google Gemini



Background level 2



Photo 1 pour créer le menu

Photo 2 pour crée le menu

Photo 3 pour crée le menu

Etoile

Obstacle 1

Obstacle 2 crée par moi

Obstacle 3

## le joueur :

Les différentes images du joueur (cycles de course, saut, glissade) ont été créées avec **Google Gemini**, puis assemblées en spritesheet.



**Remerciements** — Nous remercions **Pr. R. HANNANE** pour son encadrement tout au long du module de POO en C++, ainsi que la **Faculté des Sciences Semlalia — Université Cadi Ayyad** pour le cadre de formation offert.